

Guilherme Henrique Bernardi

Gustavo Franco Pereira

Victor Hugo Camargo

Manutenção Predial

Limeira – SP

2026

SUMÁRIO

1.	INTRODUÇÃO	1
2.	OBJETIVO	2
2.1.	<i>Objetivos Específicos</i>	2
3.	RESULTADOS ESPERADOS	3
4.	LEVANTAMENTO DE REQUISITOS	4
4.1.	<i>Requisitos Funcionais</i>	4
4.2.	<i>Requisitos Não Funcionais</i>	5
4.3.	<i>Priorização</i>	6
5.	METODOLOGIA (SCRUM)	8
5.1.	<i>Por que utilizar o Scrum no projeto "PredialFix"?</i>	8
6.	DIAGRAMAS	10
6.1.	<i>Diagrama de Classe</i>	10
6.2.	<i>Diagrama de Sequência</i>	11
7.	PROTOTIPAGEM	12
8.	BANCO DE DADOS	15
8.1.	<i>Modelo Conceitual</i>	15
8.2.	<i>Modelo Lógico</i>	16
9.	TESTES E ACESSIBILIDADE	17
9.1.	<i>Cenários de Validação por Perfil</i>	17
9.2.	<i>Responsividade e Diretrizes de Design</i>	17
10.	RECURSOS E QUALIDADE DE SOFTWARE	18
10.1.	<i>Dimensões na Qualidade e API do Sistema</i>	18
11.	CORREÇÃO DA APLICAÇÃO E PROJETO	19
11.1.	<i>Pontos de Atenção e Resolução de Inconsistências</i>	19
12.	DESENVOLVIMENTO DE APLICAÇÃO	20
12.1.	<i>ESCOLHA DE SCAFFOLDING (BREEZE VS. FILAMENT)</i>	20
12.2.	<i>PASSOS INICIAIS PARA A CRIAÇÃO DA APLICAÇÃO</i>	20
13.	COMPONENTES E ROTAS	21
13.1.	<i>COMPONENTES PRINCIPAIS DA ARQUITETURA</i>	21
13.2.	<i>DEFINIÇÃO DE ROTAS RESTFUL</i>	21

14.	DOCUMENTAÇÃO E PROCEDIMENTOS DA APLICAÇÃO	23
14.1.	<i>BASE DE CONHECIMENTO E ARTEFATOS DO PROJETO</i>	<i>23</i>
14.2.	<i>PADRÕES DE API</i>	<i>23</i>

1. INTRODUÇÃO

A manutenção predial eficiente é um dos pilares fundamentais para o pleno funcionamento de instituições educacionais de grande porte. No cenário atual do SENAI, a gestão da infraestrutura enfrenta um desafio crítico: a alta demanda por reparos, que variam de intervenções simples, como a troca de lâmpadas, a manutenções hidráulicas complexas, esbarra em um gargalo de comunicação lenta e falta de rastreabilidade. Atualmente, o fluxo de centenas de solicitações mensais carece de transparência, resultando em insatisfação entre funcionários e alunos, que não possuem visibilidade sobre o status de seus pedidos.

O projeto PredialFix surge como a resposta tecnológica a esse desafio. Focado no desenvolvimento de uma infraestrutura de Back-End robusta, o sistema visa transformar o fluxo desorganizado de solicitações em um processo de trabalho otimizado. Através de uma API segura e um servidor centralizado, o PredialFix oferecerá transparência total, permitindo que cada chamado seja acompanhado em tempo real, desde a abertura até a resolução definitiva. Mais do que um simples registro, a plataforma atuará como o cérebro da manutenção, sendo capaz de priorizar urgências e garantir que a infraestrutura da instituição suporte, com qualidade, as atividades de ensino e trabalho.

2. OBJETIVO

Desenvolver o Back-End da plataforma PredialFix, focado em uma infraestrutura de servidor robusta e em uma API segura que centralize a gestão de manutenção predial do SENAI, garantindo transparência total desde a abertura do chamado até a sua resolução final.

2.1. Objetivos Específicos

- **Gestão de Acesso e Segurança:** Sistema de autenticação seguro que permite a identificação de funcionários e alunos, garantindo a integridade dos dados e o registro de quem abriu cada solicitação.
- **Abertura e Categorização de Chamados:** Interface para descrever o problema (hidráulico, elétrico, estrutural), anexar evidências e localizar o incidente (bloco/sala).
- **Fluxo de Trabalho Transparente (Workflow):** Lógica de estados do chamado (Aberto, Em Análise, Em Execução, Concluído) que permite ao usuário acompanhar cada etapa em tempo real, eliminando a falta de feedback.
- **Algoritmo de Priorização de Urgências:** Classificação automática da criticidade dos pedidos, vazamentos graves têm prioridade sobre lâmpadas queimadas, organizando a fila de trabalho da equipe de manutenção de forma eficiente.
- **Histórico e Relatórios de Performance:** Armazenamento completo de dados para que a gestão visualize o tempo médio de atendimento e os problemas mais frequentes, facilitando decisões estratégicas de infraestrutura.

3. RESULTADOS ESPERADOS

A implementação do PredialFix deverá transformar a gestão de manutenção predial do SENAI, substituindo o atual modelo por um sistema preditivo e de alta visibilidade. Espera-se que o projeto proporcione uma redução drástica no tempo de resposta aos incidentes, uma vez que a infraestrutura de Back-End permitirá a triagem e priorização automatizada das demandas. A transparência será o resultado mais perceptível para a comunidade escolar; ao permitir que alunos e funcionários acompanhem em tempo real cada etapa do processo, da abertura à resolução, a plataforma eliminará a incerteza e as reclamações por falta de feedback. Com a adição do aplicativo mobile desenvolvido em Flutter, essa acessibilidade é ampliada: qualquer chamado poderá ser aberto ou consultado diretamente pelo smartphone, sem necessidade de acesso a um computador. Por fim, a segurança garantida pela API autenticada assegurará a integridade dos dados, gerando um histórico confiável que servirá de base para auditorias e decisões estratégicas da diretoria.

4. LEVANTAMENTO DE REQUISITOS

A definição dos requisitos do PredialFix foca em transformar as necessidades operacionais do SENAI em uma infraestrutura de Back-End lógica e funcional. Este levantamento é o que garante que o sistema não seja apenas um canal de registros, mas uma ferramenta ativa de gestão que elimina falhas de comunicação e organiza o fluxo de trabalho da manutenção.

4.1. Requisitos Funcionais

Os Requisitos Funcionais descrevem o que o sistema deve fazer, definindo as ações e funcionalidades diretas disponíveis aos usuários, como a abertura de chamados, o sistema de login e a atualização de status. Eles servem para garantir que todas as demandas operacionais do SENAI sejam atendidas, transformando as regras de negócio em recursos práticos que resolvem diretamente o problema da desorganização e da falta de transparência.

- RF01 - Autenticação e Gestão de Perfis: O sistema deve possuir um módulo de autenticação seguro que diferencie, no mínimo, dois níveis de acesso: Usuário Comum (funcionários/alunos que apenas abrem e acompanham seus chamados) e Responsável/Técnico (equipe de manutenção/chefes que atualizam e gerenciam os chamados). O login deve funcionar tanto na plataforma web quanto no aplicativo mobile.
- RF02 - Abertura de Chamados e Priorização: O sistema deve disponibilizar um endpoint para registro de novos chamados, sendo obrigatório o envio das seguintes informações: Tipo de problema (Elétrica, Hidráulica, Outros), Descrição detalhada da ocorrência, Localização exata e Nível de Urgência.
- RF03 - Máquina de Estados (Workflow): O sistema deve gerenciar o ciclo de vida do chamado, permitindo a transição estrita entre os seguintes status: Aberto → Em Análise → Em Execução → Concluído.
- RF04 - Filtragem e Listagem: O sistema deve permitir que os usuários filtrem e visualizem a lista de chamados com base em seus status atuais, localização e nível de urgência, tanto pela interface web quanto pelo aplicativo mobile.
- RF05 - Regra de Negócio de Atualização: O sistema deve bloquear a alteração de status de um chamado (ex: mover para "Em Execução") caso o usuário logado não tenha o perfil de "Responsável" ou não seja o chefe do setor correspondente.
- RF06 - Histórico de Manutenção por Unidade: O sistema deve armazenar e permitir a consulta de todo o histórico de serviços já realizados, filtráveis por um espaço específico ou área comum (ex: "Laboratório de Informática 3"), para facilitar a análise de reincidências.

- RF07 - Transparência e Notificações (Feedback): O sistema deve disparar atualizações de progresso para o criador do chamado a cada mudança de status (ex: "Em Análise", "Técnico a caminho", "Serviço Concluído"), garantindo a total transparência exigida pelo Senai.
- RF08 - Aplicativo Mobile (Abertura e Acompanhamento): O sistema deve disponibilizar um aplicativo mobile desenvolvido em Flutter/Dart, compatível com Android e iOS, que permita ao usuário: realizar login com suas credenciais, abrir novos chamados preenchendo todos os campos obrigatórios, acompanhar em tempo real o status dos chamados abertos por ele e visualizar o histórico de chamados anteriores.
- RF09 - Notificações Push (Mobile): O aplicativo mobile deve enviar notificações push ao usuário sempre que houver alteração de status em um chamado de sua autoria, mesmo que o aplicativo esteja em segundo plano ou fechado.

4.2. Requisitos Não Funcionais

Os Requisitos Não Funcionais determinam como o sistema deve se comportar sob a ótica técnica, estabelecendo critérios rigorosos de qualidade, como segurança, tempo de resposta e escalabilidade. Eles são essenciais para assegurar que a infraestrutura da API seja capaz de suportar um alto volume de acessos simultâneos sem lentidão, protegendo os dados institucionais contra falhas e garantindo alta disponibilidade para que o sistema esteja sempre acessível durante as emergências prediais.

- RNF01 - (Segurança e Autenticação): A comunicação com a API deve ser criptografada via HTTPS. A autenticação das requisições deve ser feita utilizando tokens seguros (como o padrão JWT - JSON Web Tokens), garantindo que apenas usuários logados acessem os dados, tanto pela web quanto pelo aplicativo mobile.
- RNF02 - (Desempenho): O servidor back-end deve ser capaz de processar as requisições de consulta e abertura de chamados com um tempo de resposta médio inferior a 2 segundos, mesmo lidando com as centenas de solicitações mensais da unidade.
- RNF03 - (Disponibilidade): O sistema deve operar com alta disponibilidade (meta de uptime de 99%), garantindo que problemas estruturais possam ser relatados por alunos e funcionários a qualquer momento de funcionamento do Senai.
- RNF04 - (Arquitetura e Integração): O back-end deve ser construído seguindo os padrões de uma API RESTful, retornando dados estruturados em formato JSON, garantindo a integração tanto com a interface web quanto com o aplicativo mobile Flutter do PredialFix.

- RNF05 - (Confiabilidade / Integridade de Dados): O sistema deve utilizar um banco de dados relacional (ex: PostgreSQL, MySQL) robusto, garantindo a integridade referencial entre os chamados, os usuários e os locais do prédio, sem perda de dados em caso de falhas nas transações.
- RNF06 - (Documentação): A API desenvolvida deve ser documentada de forma clara (utilizando ferramentas como Swagger/OpenAPI), descrevendo rotas, parâmetros e respostas esperadas, facilitando o trabalho das equipes responsáveis pela interface web e pelo aplicativo mobile.
- RNF07 - (Compatibilidade Mobile): O aplicativo mobile deve ser compatível com dispositivos Android (versão 8.0 ou superior) e iOS (versão 13 ou superior), garantindo cobertura para a maior parte dos dispositivos em uso pela comunidade escolar.
- RNF08 - (Usabilidade Mobile): A interface do aplicativo mobile deve ser responsiva e adaptada para diferentes tamanhos de tela, seguindo as diretrizes de usabilidade do Material Design (Android) e Human Interface Guidelines (iOS), garantindo uma experiência intuitiva para todos os perfis de usuário.

4.3. Priorização

Para priorizar os requisitos do PredialFix de forma estratégica, foi utilizado o método MoSCoW. Essa técnica ajuda a focar no que é vital para o funcionamento da API e o que pode ser deixado para fases posteriores.

- **MUST HAVE (DEVE TER)**
São as funcionalidades inegociáveis para que o sistema exista. Sem elas, o projeto não atende ao objetivo principal de organizar os chamados.
- **SHOULD HAVE (DEVERIA TER)**
Funcionalidades de alta prioridade que devem ser incluídas, mas cuja ausência não impede o sistema de funcionar em curto prazo.
- **COULD HAVE (PODERIA TER)**
Recursos que agregariam valor e melhorariam a experiência do usuário, mas que só serão desenvolvidos se houver tempo e recursos sobrando.
- **WON'T HAVE (NÃO TERÁ)**
Funcionalidades que foram discutidas, mas que estão fora do escopo desta entrega inicial.

REQUISITO	PRIORIZAÇÃO	EXPLICAÇÃO
RF01, RF02, RF03	Must have	Essenciais para a funcionalidade básica do sistema: autenticação, abertura de chamados e gerenciamento do fluxo de trabalho.
RNF01, RNF05	Must have	Críticos para segurança dos dados e integridade das informações no sistema.
RF05, RF07	Must have	Garantem regras de negócio essenciais e transparência no acompanhamento dos chamados.
RF08	Must have	O aplicativo mobile é parte central do escopo do projeto, sendo obrigatório para garantir o acesso à plataforma via smartphone.
RF04, RF06	Should have	Importantes para facilitar a gestão e consulta de chamados, permitindo análise de reincidências.
RNF02, RNF03	Should have	Desempenho e disponibilidade são importantes para a experiência do usuário e confiabilidade do sistema.
RF09, RNF07	Should have	Notificações push e compatibilidade mobile ampliam a experiência do usuário no app, mas não bloqueiam o funcionamento inicial.
RNF04, RNF06	Could have	Facilitam integração futura e manutenção, mas não são bloqueadores para o funcionamento inicial.
RNF08	Could have	Refinamentos de usabilidade e aderência a guidelines de design mobile agregam qualidade, mas podem ser evoluídos ao longo do projeto.

5. METODOLOGIA (SCRUM)

O Scrum é um framework (ou estrutura de trabalho) ágil focado na gestão e no desenvolvimento de projetos complexos. Em vez de tentar construir todo o sistema de uma só vez e entregá-lo apenas no final de meses de trabalho, o Scrum divide o projeto em ciclos curtos e contínuos chamados Sprints (que geralmente duram de 1 a 4 semanas).

Ao final de cada Sprint, a equipe entrega uma parte do produto que já funciona e agrega valor. Ele é baseado em três pilares fundamentais: Transparência, Inspeção e Adaptação, e organiza a equipe através de papéis bem definidos (Product Owner, Scrum Master e Desenvolvedores) e cerimônias rápidas (Planejamento, Reunião Diária, Revisão e Retrospectiva).

5.1. Por que utilizar o Scrum no projeto "PredialFix"?

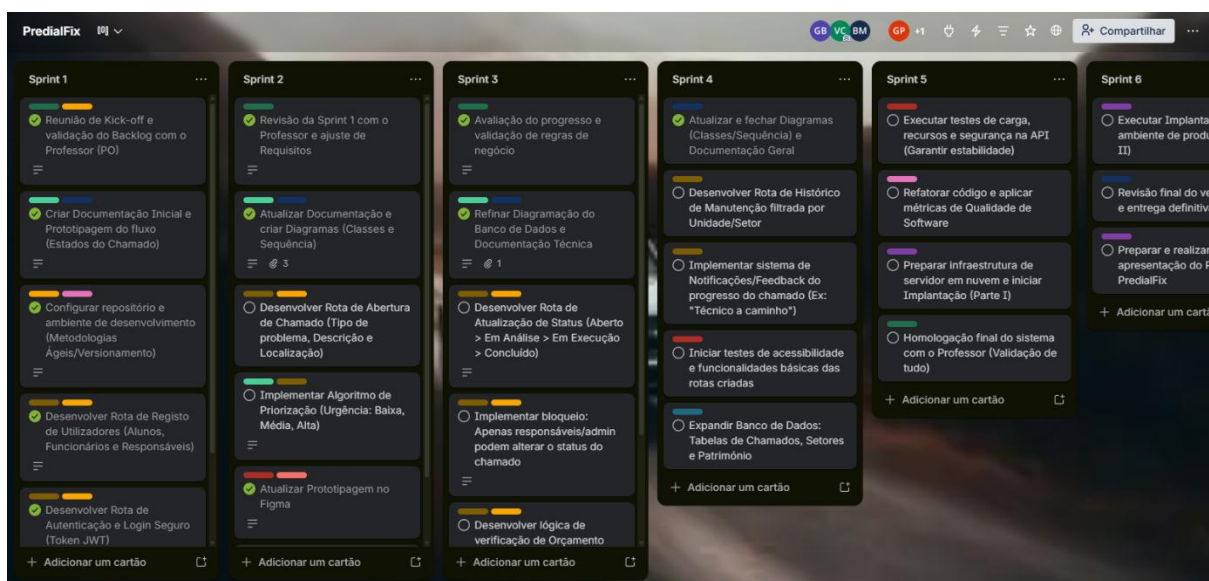
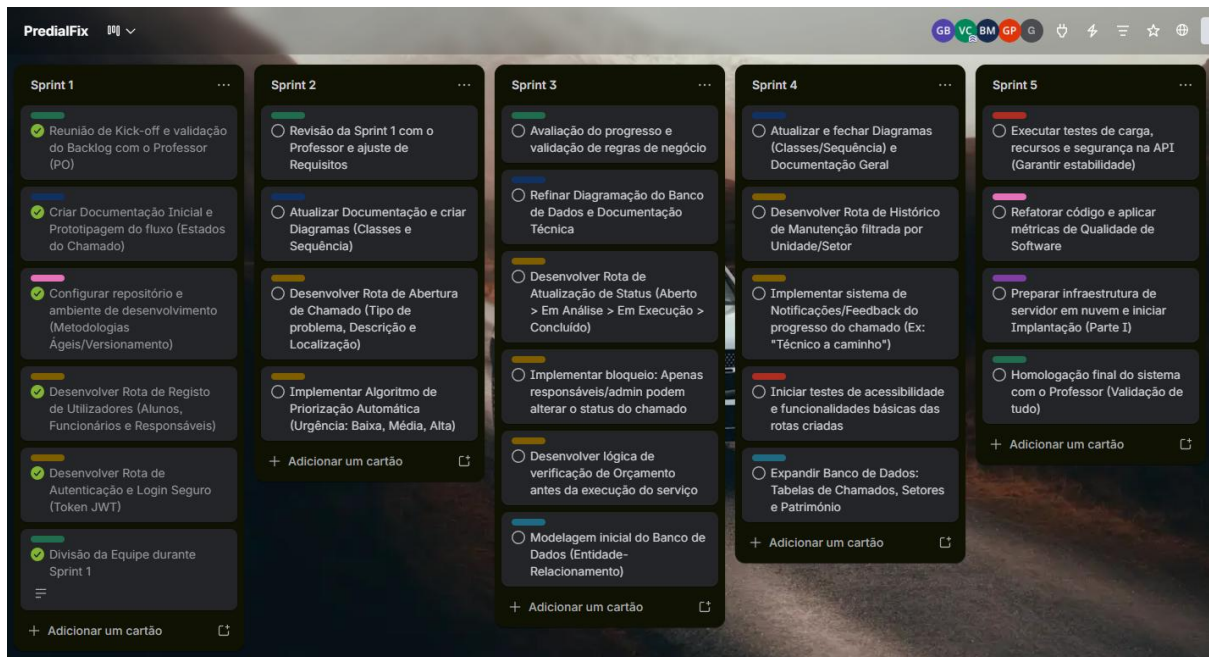
O cenário do SENAI exige uma solução rápida, robusta e que resolva o problema da "falta de transparência e demora no atendimento". O Scrum é a escolha ideal para esse desafio pelos seguintes motivos:

- **Entregas Incrementais e Rápidas (Sprints):** Sua equipe não precisa entregar a API inteira de uma vez. Vocês podem usar as Sprints para entregar as Funcionalidades Essenciais em partes. Por exemplo: a Sprint 1 pode focar apenas na "Gestão de Usuários" (Autenticação no Laravel). A Sprint 2 foca na "Abertura de Chamados" e no banco de dados MySQL. Isso permite que o SENAI comece a testar o núcleo do sistema muito mais rápido.
- **Priorização Inteligente (Product Backlog):** O desafio fala em "priorizar urgências". No Scrum, os requisitos (as 5 funcionalidades essenciais) formam o Product Backlog, uma lista ordenada. O que é mais crítico para o SENAI resolver hoje? Provavelmente o registro do problema e o Workflow. O Scrum garante que a equipe foque primeiro no que traz mais impacto para os funcionários e alunos.
- **Transparência e Alinhamento Constante:** O problema central do SENAI é a falta de transparência nos chamados. Ironicamente, o Scrum resolve a falta de transparência no desenvolvimento. Com as reuniões diárias (Daily Scrum), a equipe de Back-End saberá exatamente quem está cuidando da validação de dados (Request Validation), quem está criando as rotas RESTful e quem está travado em algum problema com o Eloquent ORM.
- **Adaptação às Regras de Negócio:** Modelar os status dos chamados (Aberto -> Em Análise -> Em Execução -> Concluído) e a lógica de quem pode atualizar o quê (multi-nível) pode gerar dúvidas durante o desenvolvimento. O Scrum permite que, ao final de cada Sprint (na Revisão), a equipe valide com os interessados se a API está se comportando como esperado, permitindo ajustes rápidos antes de avançar para as "Notificações de Progresso" ou "Histórico".

- Foco na Qualidade Técnica: Como há requisitos rigorosos (validação, respostas em JSON, documentação clara), as Sprints permitem que a equipe estabeleça uma "Definição de Pronto" (Definition of Done). Uma funcionalidade só avança se a rota estiver criada, testada, validada e com o código devidamente subido no GitHub.

Ao usar o Scrum, o desenvolvimento do PredialFix deixa de ser uma caixa preta demorada e passa a ser um processo previsível, focado em resolver a dor real do SENAI etapa por etapa.

Foi utilizado o Trello para a parte de metodologia: [Trello](#)



6. DIAGRAMAS

Os diagramas são as representações visuais que estruturam a lógica do sistema, funcionando como uma ponte entre os requisitos de negócio e a implementação técnica. Eles permitem que a arquitetura da API seja compreendida de forma clara, garantindo que o fluxo de informações entre o usuário e o servidor ocorra de maneira segura e organizada.

6.1. Diagrama de Classe

Mapeia a estrutura estática dos dados, definindo as entidades principais, seus atributos e as relações de cardinalidade. É essencial para garantir a integridade do banco de dados e a organização das informações.

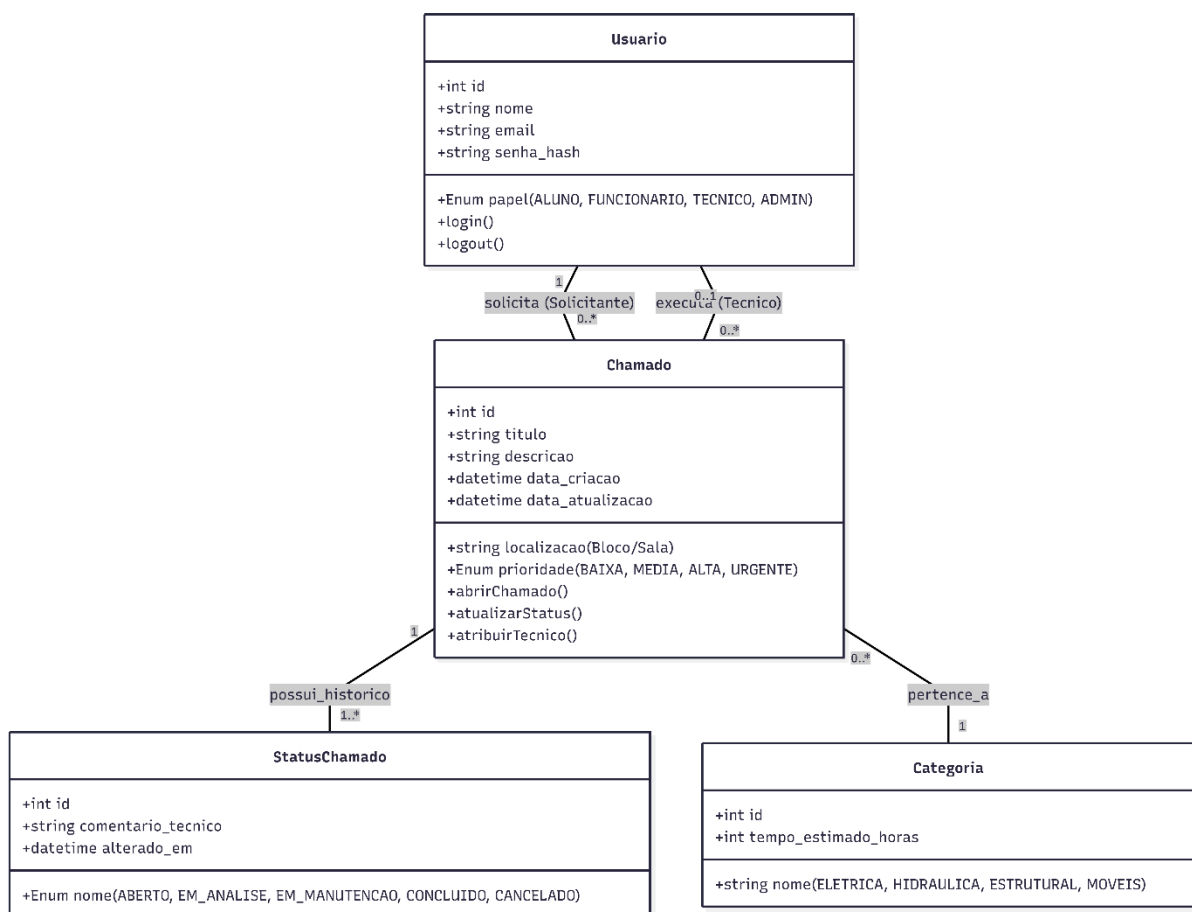


Diagrama completo no link: [Diagrama de Classe](#)

6.2. Diagrama de Sequência

Descreve o comportamento dinâmico do sistema, ilustrando a ordem cronológica das interações entre o usuário, a API e o banco de dados. Validando fluxos críticos, como a abertura de chamados, garante que as regras de segurança e priorização sejam aplicadas antes da persistência dos dados.

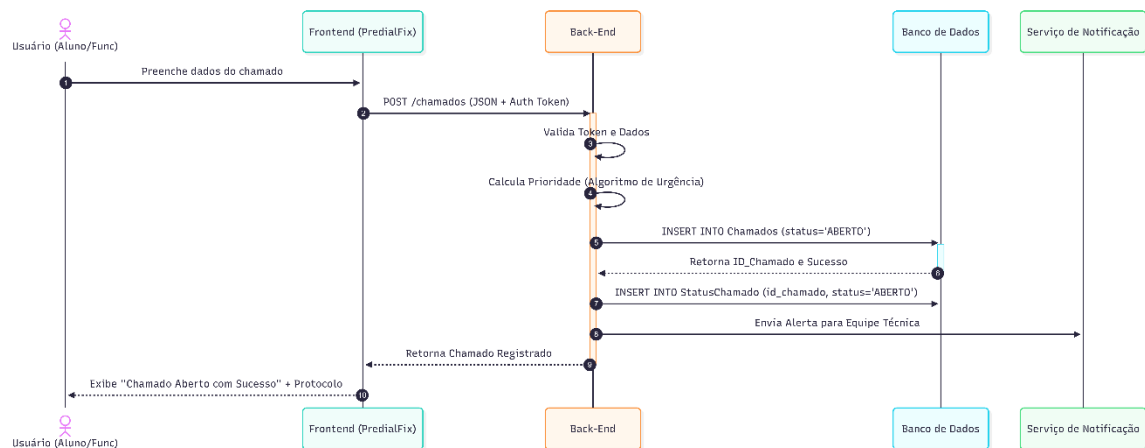


Diagrama completo no link: [Diagrama de Sequência](#)

7. PROTOTIPAGEM

Foi utilizado o Figma para o desenvolvimento e prototipagem do aplicativo: [Figma](#)

Login



Menu

SENAI

Home

Novo Chamado

Gerenciar Chamados

Sair

Chamados Feitos

32

Chamados em Andamento

32

Chamados Feitos

32

Chamados Feitos

32

Chamados Recentes

Tipo	Descrição	Local	Data de Abertura	Status	Data de Término
Elétrica	Tomada em Curto...	Bloco A, Sala 1	02/03/2026	Em Andamento	02/03/2026
Elétrica	Tomada em Curto...	Bloco A, Sala 1	02/03/2026	Em Andamento	02/03/2026
Hidráulica	Tomada em Curto...	Bloco A, Sala 1	02/03/2026	Em Andamento	02/03/2026
Elétrica	Tomada em Curto...	Bloco A, Sala 1	02/03/2026	Em Andamento	02/03/2026

Relatar novo Problema

Registrar Chamado

SENAI Home Novo Chamado Gerenciar Chamados Sair 

Olá, [Nome do Professor]. Relate o problema abaixo.

Tipo de Incidente:

▼ Selecione

Local

Mapa

Seção Técnica

▼ Selecione

 (Elétrica, Hidráulica, Civil, etc)

Nível de Prioridade

▼ Selecione

 (Baixa, Média, Alta, Crítica)

Nível de Complexidade

▼ Selecione

 (Simples, Média, Complexa)

Tipo de Trabalho

▼ Selecione

 (Preventiva, Corretiva, Melhoria)

Foto

Faça upload

ou arraste uma imagem

Enviar

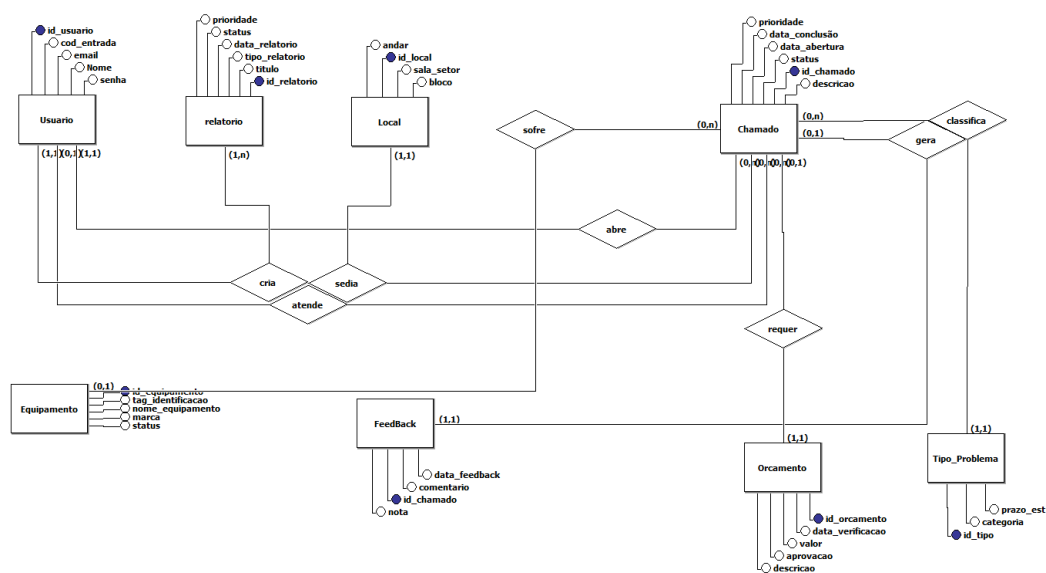
8. BANCO DE DADOS

O banco de dados atuará como o núcleo de armazenamento seguro e centralizado do PredialFix. Sua principal função é registrar e persistir todas as informações críticas da instituição, desde as credenciais e perfis de acesso dos usuários até os detalhes e o histórico imutável de cada chamado de manutenção.

Foi utilizado o BrModelo para a modelagem do banco de dados do aplicativo.

8.1. Modelo Conceitual

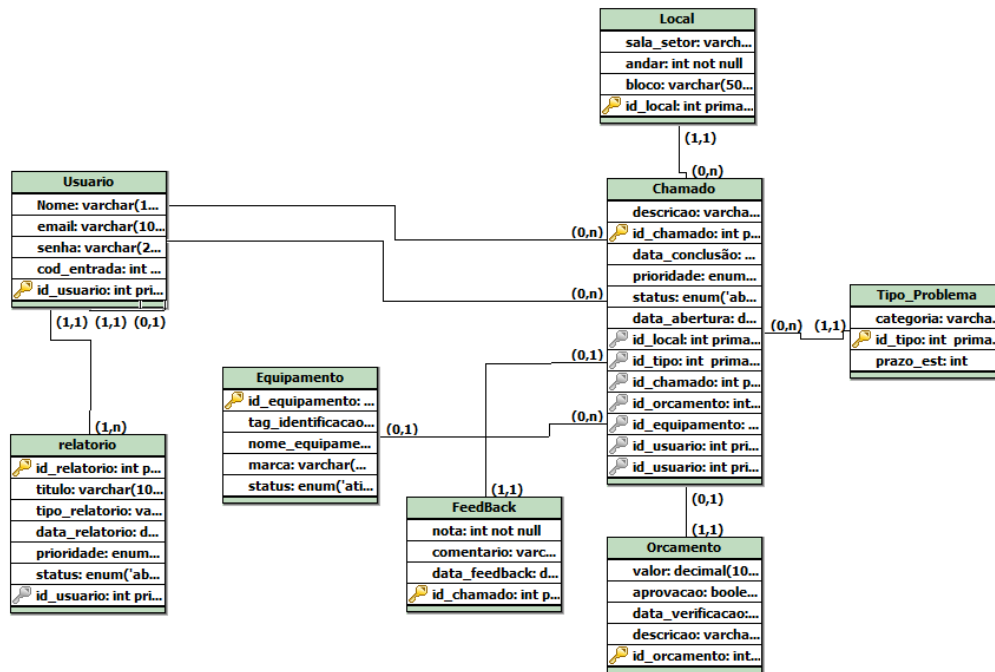
O Modelo Conceitual ilustra a visão do sistema de gestão de chamados. Ele mapeia as principais entidades do projeto e as regras de negócio que definem como essas informações interagem entre si no dia a dia.



Modelo completo no link: [Modelo Conceitual](#)

8.2. Modelo Lógico

O Modelo Lógico detalha a estrutura de tabelas que será implementada no banco de dados.



Modelo completo no link: [Modelo Lógico](#)

9. TESTES E ACESSIBILIDADE

A etapa de testes é fundamental para assegurar que o sistema PredialFix se comporte de forma correta, segura e acessível para todos os perfis de usuário previstos — Administrador, Gerente de Manutenção, Técnico, Professor e Aluno. Os testes abrangem desde a verificação das permissões de acesso por nível de perfil até a validação das funcionalidades centrais da plataforma, como abertura de chamados, atualização de status e avaliação de serviços concluídos.

Para a execução dos testes, devem ser seguidos os procedimentos descritos no documento `GUIA_TESTE_ALUNOS.md`, que contempla oito testes detalhados com passos, observações e resultados esperados. A preparação do ambiente inclui a carga dos dados de teste por meio do comando abaixo:

```
php artisan db:seed --class=UsersSeeder
```

9.1. Cenários de Validação por Perfil

Os principais cenários validados durante os testes são:

- Autenticação por perfil: verificar que cada nível de acesso realiza login com as credenciais corretas e é redirecionado para as telas correspondentes às suas permissões.
- Acessibilidade funcional do Professor: confirmar que o usuário com perfil Aluno consegue acessar o dashboard, criar novos chamados e editar chamados próprios com status "Aberto", sem acesso a campos técnicos ou alteração de status.
- Restrição de operações indevidas: validar que tentativas de alterar status por perfis não autorizados são bloqueadas pelo sistema, retornando mensagens de erro adequadas.
- Avaliação de chamados concluídos: verificar que o usuário solicitante consegue avaliar um chamado após sua conclusão.
- Testes de banco de dados e logs: checar a integridade dos registros persistidos e confirmar as entradas nos logs de auditoria após cada operação sensível.

9.2. Responsividade e Diretrizes de Design

A verificação de acessibilidade também contempla a responsividade da interface para diferentes tamanhos de tela, seguindo as diretrizes do Material Design para dispositivos Android e as Human Interface Guidelines para iOS, garantindo uma experiência consistente em toda a comunidade escolar.

10. RECURSOS E QUALIDADE DE SOFTWARE

A qualidade de software no PredialFix é assegurada por meio da adoção de boas práticas de desenvolvimento, validação contínua e definição criteriosa dos critérios de aceitação de cada funcionalidade. O projeto segue a metodologia Scrum, na qual cada Sprint possui uma Definição de Pronto (Definition of Done): uma funcionalidade só é considerada concluída quando a rota está criada, testada, validada e o código está devidamente versionado no repositório.

10.1. *Dimensões na Qualidade e API do Sistema*

Os recursos de qualidade implementados ou planejados para o sistema abrangem as seguintes dimensões:

- Validação de requisições (Request Validation): todas as entradas de dados enviadas à API são validadas antes de qualquer operação no banco de dados, prevenindo registros inconsistentes e falhas de integridade.
- Autorização por Policies (Laravel Eloquent Policies): o controle de acesso a cada operação é definido em classes de Policy dedicadas — como ChamadoPolicy —, separando a lógica de negócio da lógica de autorização e facilitando auditorias e manutenções futuras.
- Padronização de respostas: a API retorna dados estruturados em formato JSON, com códigos HTTP semânticos, garantindo previsibilidade para os clientes web e mobile.
- Cobertura de testes: os fluxos críticos, como abertura de chamados e transições de status, são cobertos por testes documentados no GUIA_TESTE_ALUNOS.md, com resultados esperados definidos para cada cenário.
- Versionamento e rastreabilidade: todo o código é mantido no GitHub, permitindo histórico de alterações, revisão por pares (code review) e rollback em caso de regressão.
- Documentação da API: as rotas, parâmetros e respostas esperadas são documentados de forma clara, utilizando ferramentas como Swagger/OpenAPI, em conformidade com o requisito não funcional RNF06.

A combinação dessas práticas visa reduzir a incidência de bugs em produção, facilitar a integração de novos membros ao time e garantir que o sistema suporte com estabilidade o volume de centenas de solicitações mensais do SENAI.

11. CORREÇÃO DA APLICAÇÃO E PROJETO

Esta etapa contempla a identificação, registro e resolução de inconsistências encontradas tanto na aplicação individual quanto no projeto como um todo, abrangendo correções de lógica de negócio, ajustes de interface e refinamentos na camada de autorização.

As correções são organizadas por prioridade e registradas no backlog do projeto (gerenciado via Trello), sendo endereçadas nas Sprints subsequentes à identificação.

11.1. *Pontos de Atenção e Resolução de Inconsistências*

Os principais pontos de atenção mapeados são:

- Exibição indevida de campos técnicos para o perfil Aluno: a view `resources/views/chamados/create.blade.php` deve ser revisada para garantir que campos como tipo de serviço técnico e estimativa de tempo sejam ocultados via diretivas Blade (`@can`, `@unless`) para perfis sem essa permissão.
- Permissão incorreta de alteração de status: o método `updateStatus()` no `ChamadoController.php` deve ser verificado para assegurar que somente perfis de Responsável ou Técnico autorizados consigam transitar o chamado entre os estados do workflow (Aberto → Em Análise → Em Execução → Concluído).
- Inconsistências de autenticação entre plataformas: validar que o fluxo de login funciona de forma equivalente tanto na interface web quanto no aplicativo mobile Flutter, respeitando a autenticação via JWT (JSON Web Tokens) conforme o requisito RNF01.
- Correções de integridade referencial no banco de dados: revisar as constraints entre as tabelas de chamados, usuários e locais para prevenir registros órfãos ou violações de chave estrangeira.
- Ajustes de usabilidade na interface mobile: corrigir comportamentos de layout em telas de tamanho reduzido, garantindo conformidade com as diretrizes do requisito RNF08.

Cada correção aplicada deve ser acompanhada de reteste do cenário afetado, com registro do resultado no guia de testes, assegurando que a resolução não introduza regressões em funcionalidades previamente aprovadas.

12. DESENVOLVIMENTO DE APLICAÇÃO

O desenvolvimento do PredialFix tem como base o framework Laravel, com a criação da aplicação podendo ser iniciada com o scaffolding do Laravel Breeze ou com o painel administrativo do Laravel Filament, a depender das necessidades de interface e do perfil de desenvolvimento do time.

12.1. ESCOLHA DE SCAFFOLDING (BREEZE VS. FILAMENT)

- Laravel Breeze é a opção indicada para equipes que desejam construir a interface do zero, com maior controle sobre as views Blade, autenticação simples e estrutura enxuta. É adequado para o front-end web do PredialFix, onde a personalização das telas de login, dashboard e chamados é necessária.
- Laravel Filament é recomendado para a construção ágil de painéis administrativos, oferecendo componentes prontos para tabelas, formulários, filtros e permissões, o que acelera o desenvolvimento das telas de gestão utilizadas por Administradores e Gerentes de Manutenção.

12.2. PASSOS INICIAIS PARA A CRIAÇÃO DA APLICAÇÃO

Os passos iniciais para criação da aplicação são:

- Instalação do Laravel e configuração do ambiente: configurar o servidor local (Laravel Sail ou Laragon), banco de dados MySQL/PostgreSQL e variáveis de ambiente no arquivo .env.
- Scaffolding de autenticação: executar o Breeze ou Filament para gerar as rotas, controllers e views de autenticação base, sobre as quais o sistema de níveis de acesso será construído.
- Configuração dos modelos e migrações: criar os modelos Eloquent (User, Chamado, Local) e as respectivas migrações, refletindo o Modelo Lógico definido durante a fase de banco de dados.
- Seed de dados iniciais: popular o banco com os perfis de usuário de teste via UsersSeeder, permitindo validação imediata das funcionalidades.
- Inicialização do repositório: versionar o projeto no GitHub desde o primeiro commit.

Esta etapa marca a transição do projeto da fase de planejamento e documentação para a fase de implementação efetiva, sendo o ponto de partida para o desenvolvimento incremental orientado pelas Sprints do Scrum.

13. COMPONENTES E ROTAS

A arquitetura do PredialFix é estruturada em torno de componentes bem definidos, modelos, controllers, policies e views, e de rotas RESTful que expõem as funcionalidades da API para os clientes web e mobile.

13.1. COMPONENTES PRINCIPAIS DA ARQUITETURA

Os componentes principais do sistema são:

- **Models:** representam as entidades do sistema (User, Chamado, Local) e encapsulam as regras de negócio e os relacionamentos Eloquent. O modelo User é responsável por armazenar o nível de acesso (role) de cada usuário, sendo consultado pelas Policies em toda requisição de autorização.
- **Controllers:** recebem as requisições HTTP, aplicam as validações necessárias, delegam a lógica às Policies e retornam as respostas em JSON. O ChamadoController é o componente central, gerenciando o ciclo de vida completo dos chamados.
- **Policies:** classes de autorização (como ChamadoPolicy) que determinam, com base no perfil do usuário autenticado, quais operações são permitidas sobre cada recurso. São registradas no AuthServiceProvider e invocadas via `$this->authorize()` nos controllers.
- **Views (Blade):** templates responsáveis pela renderização da interface web, utilizando diretivas como `@can` e `@cannot` para exibir ou ocultar elementos conforme as permissões do usuário logado.
- **Seeders:** classes responsáveis pela população do banco de dados com dados de teste, facilitando o desenvolvimento e a validação em ambiente local.

13.2. DEFINIÇÃO DE ROTAS RESTFUL

As rotas do sistema seguem os padrões RESTful e são definidas nos arquivos `routes/web.php` (interface web) e `routes/api.php` (API para o aplicativo mobile). As principais rotas do módulo de chamados são:

Método	URI	Ação	Permissão
GET	/chamados	Listar chamados	Todos os perfis autenticados
GET	/chamados/create	Formulário de novo chamado	Todos os perfis autenticados
POST	/chamados	Registrar novo chamado	Todos os perfis autenticados
GET	/chamados/{id}/edit	Formulário de edição	Dono do chamado (status Aberto)
PUT	/chamados/{id}	Atualizar chamado	Dono do chamado (status Aberto)
PATCH	/chamados/{id}/status	Atualizar status	Responsável / Técnico autorizado
POST	/chamados/{id}/avaliar	Avaliar chamado	Dono do chamado (status Concluído)

As rotas da API são protegidas pelo middleware auth:sanctum (ou JWT), garantindo que apenas requisições autenticadas acessem os recursos, em conformidade com o requisito de segurança RNF01.

14. DOCUMENTAÇÃO E PROCEDIMENTOS DA APLICAÇÃO

A documentação do PredialFix é mantida de forma contínua e organizada em múltiplos documentos especializados, cada um direcionado a um perfil específico de leitura. O índice central (INDICE_DOCUMENTACAO.md) serve como ponto de entrada e mapeia todos os documentos disponíveis, orientando novos membros e consultores sobre onde encontrar cada informação.

14.1. *BASE DE CONHECIMENTO E ARTEFATOS DO PROJETO*

Os documentos que compõem a base de conhecimento do projeto são:

- RESUMO_SISTEMA.md: visão geral executiva do sistema, descrevendo o que foi implementado, os níveis de acesso, os arquivos criados ou modificados e as permissões resumidas de cada perfil. Recomendado como primeira leitura para qualquer novo membro do projeto.
- NIVEIS_ACESSO.md: documentação completa do sistema de autorização, detalhando as permissões de cada perfil, a implementação técnica das Políticas e exemplos de código comentado.
- MANIFESTO_MUDANCAS.md: relatório técnico das alterações realizadas, listando arquivos modificados e criados, impacto por arquivo, testes realizados e próximos passos planejados.

14.2. *PADRÕES DE API*

A documentação da API segue o padrão Swagger/OpenAPI (requisito RNF06), descrevendo todas as rotas, parâmetros obrigatórios e opcionais, tipos de dados esperados e exemplos de resposta em JSON. Esse formato garante que as equipes responsáveis pela interface web e pelo aplicativo mobile Flutter possam integrar-se à API sem ambiguidades.